Lab 05: Local Search Algorithms

HRM A A BANDARA | IT24100315 | SE3062 Intelligent Systems

Part A — Simulation Analysis & Parameter Tweaks (2.0 Marks)
1. The Hill Climbing trap and step size (0.5 marks)
Set the simulator to Hill Climbing (Greedy) with Starting X = 100.
Tweak 1: run with Step Size = 2 and note where the search becomes trapped.
Tweak 2: change Step Size = 20 and run again. Does it step over the local maximum?
Run
Final X
Peak reached (local / global)
Step Size = 2
146
Step Size = 20
140
Year 3 · Semester 1 · 2026
Local Maximum
Local Maximum

Explain the trade-off: how does a larger step size help the search escape a local optimum, and what is the dangerous drawback of setting the
step size too large as the search nears the global maximum?
A larger step size can help Hill Climbing escape a narrow local optimum because it evaluates states farther away and may jump
into a better region of the fitness landscape. However, if the step size is too large, the search becomes too coarse and can jump
over the true global maximum or fail to evaluate states close to its peak. In this simulation, Step Size 20 stopped at X = 140 with
fitness 265.29, while Step Size 2 reached X = 146 with the slightly better fitness 266.06. Therefore, the larger step did not escape
the local optimum in this run and actually produced a less precise result.
2. Initial condition sensitivity (0.5 marks)
Stay on Hill Climbing and set Step Size = 5.
Tweak 1: start at X = 50, run, and record the final peak.
Tweak 2: start at X = 850, run, and record the final peak.
Starting X
Final X
Final objective value
50
50
248.38
850
765
281.38
Observe & explain: compare the two final states. Why is greedy local search so sensitive to its initial configuration, and how does Random-
Restart Hill Climbing mitigate this specific vulnerability?
Greedy Hill Climbing is very sensitive to its initial state because it only considers neighbouring states and always moves toward
an immediate improvement. Starting at X = 50 caused the search to stop immediately at a local optimum with fitness 248.38,
while starting at X = 850 led it toward a much better peak at X = 765 with fitness 281.38. Random-Restart Hill

Climbing reduces
this vulnerability by running Hill Climbing multiple times from different randomly selected starting positions and keeping the
best result found.
3. Simulated Annealing: the cooling schedule (0.5 marks)
Switch to Simulated Annealing with Starting X = 100 and Initial Temp = 100.
Tweak 1: Cooling Rate = 0.50 (fast cooling). Run the simulation.
Tweak 2: Cooling Rate = 0.99 (slow cooling). Run the simulation.
Cooling rate
Final X / value
Behaviour observed
0.50 (fast)
best fitness = 254.85
cooled very quickly; halted after only 8
steps; little exploration.
0.99 (slow)
best fitness = 247.83
explored for 528 steps; accepted many
worse moves early; gradually became
more selective
Explain the impact: how does the cooling rate control the balance between exploration (random walk) and exploitation (greedy climbing)?
Referring to P = e^(ΔE / T) , state mathematically what happens to the acceptance probability for a worsening move when the temperature
falls too rapidly.

The cooling rate controls how long Simulated Annealing continues exploring. With a low cooling rate such as 0.50, the
temperature drops very rapidly, so the algorithm quickly stops accepting worse states and behaves more like greedy Hill
Climbing. With a high cooling rate such as 0.99, the temperature decreases slowly, so the algorithm keeps exploring for much
longer and can accept worse moves early in the search. In this run, the 0.50 setting halted after only 8 steps, while the 0.99
setting continued for 528 steps. For a worsening move, (Delta E < 0), so (P = e^{ΔE / T}). If (T) falls too quickly, (ΔE / T)
becomes a large negative number and (P ->0), meaning worse moves are almost never accepted.
4. Local Beam Search: the loss of diversity (0.5 marks)
Set the simulator to Local Beam Search with Beam Width (k) = 3 and run from a random start. You will most likely see all three agents converge
onto the same peak within a few iterations, leaving the rest of the landscape unexplored.
Observe & explain: using the algorithm's successor-selection logic, explain mathematically why this loss of diversity occurs. In your answer, state
how the successors of all k states are pooled and how the top k are chosen.
In Local Beam Search, each of the (k) current states generates successors, and all successors are placed into one shared pool. If
each state generates (m) successors, the pool contains up to (k times m) candidates. The algorithm then ranks all candidates by
fitness and selects only the top (k) states for the next iteration. Because the best successors often come from the same promising
region of the landscape, several of the selected states can become very similar or converge on the same peak. In this run, the top
states quickly became concentrated around fitness values 175, 174 and 169, showing how diversity is lost when selection always

keeps the absolute best (k) successors.
Section 2: Part B — Theoretical Foundations (MCQs) (1.5 Marks)
Select one option per question and type the letter in the answer box. 0.5 marks each.
5. Which statement best distinguishes Local Search from standard pathfinding (such as A*)?
A) Local search uses heuristics, while standard pathfinding does not.
B) Local search aims to find the optimal sequence of actions, while pathfinding only cares about the final state.
C) Standard pathfinding retains paths in memory to backtrack, whereas local search operates using only the current node (and its neighbours),
focusing on the state rather than the path.
D) Local search algorithms are guaranteed to be complete and optimal in continuous spaces.
C
6. In Simulated Annealing the probability of accepting a worse state is P = e^(ΔE / T) . As the temperature T approaches
0, what happens to the behaviour of the algorithm?
A) It behaves exactly like a purely random walk.
B) It behaves exactly like greedy Hill Climbing, accepting only better states.
C) It accepts all states regardless of their fitness evaluation.
D) The algorithm terminates automatically because ΔE becomes undefined.
B
7. How does Stochastic Beam Search solve the loss-of-diversity problem found in standard Local Beam Search?
A) By choosing the next k states randomly, with probability proportional to their fitness, rather than strictly choosing the absolute best k.
B) By increasing the beam width k dynamically whenever a local minimum is detected.
C) By forcing agents to move in opposite directions (repulsion) when they get too close to each other.
D) By completely resetting the temperature whenever all k states converge.
A

Part C: Completed implementation

The supplied PDF test runner contains only random.seed(42). The implementation below completes its generate-evaluate-prune cycle. A separate reproducible parabola check was added; it checks a tolerance of 2 around x = 50, since random successors do not guarantee the exact maximum.

```
"""Lab 05: generate, evaluate, pool and prune local beam successors."""

import random


def local_beam_search(initial_states, k, max_iters, objective_fn):
    """Maximize objective_fn while keeping the best k pooled successors."""
    current_states = initial_states
    for iteration in range(max_iters):
        all_successors = []
        for state in current_states:
            for _ in range(5):
                neighbor = state + random.uniform(-10, 10)
                fit = objective_fn(neighbor)
                all_successors.append((neighbor, fit))
        all_successors.sort(key=lambda item: item[1], reverse=True)
        current_states = [item[0] for item in all_successors[:k]]
    return max(current_states, key=objective_fn)


# The supplied PDF runner ends after random.seed(42).
# This separate check supplies the missing parabola test.
if __name__ == "__main__":
    random.seed(42)
    def objective(x):
```

```
    return -(x - 50) ** 2 + 100

best = local_beam_search([0, 25, 100], 3, 100, objective)
print(f"Best state: {best:.6f}")
print(f"Best objective value: {objective(best):.6f}")
assert abs(best - 50) < 2, "Result must be within 2 of the parabola maximum."
print("SUCCESS: Local Beam Search reached the parabola maximum within tolerance.")
```

Exact test output

```
Best state: 48.490153
Best objective value: 97.720362
SUCCESS: Local Beam Search reached the parabola maximum within tolerance.
```

Discussion: Beam search versus independent hill climbing

Local Beam Search pools successors from all three states and selects the best three overall. All selected states can descend from one parent and converge on the same region. Three independent Hill Climbing runs select a neighbour separately for each run, so one run cannot replace the others. Independent runs generally retain more diversity, although they may still reach the same peak.

Discussion: Expensive objective with a fixed evaluation budget

I would choose Simulated Annealing, which evaluates one proposed neighbour per iteration. I would enforce an evaluation counter and stop when the budget is exhausted, using a moderate initial temperature and a cooling rate matched to that budget. For B moves and desired final temperature Tf, choose alpha = (Tf / T0) ** (1 / B). The recorded 0.50 run stopped after 8 steps, whereas 0.99 explored for 528 steps. Rapid cooling restricts exploration; very slow cooling can consume the budget before sufficient exploitation. Local Beam Search with k = 3 evaluates 15 successors per iteration, making it more costly for the same iteration count. The observations are individual stochastic runs, so they do not establish that slow cooling always yields a better result.

GitHub: https://github.com/ashenbandara02/IT3012---Practical-Base/tree/Week_05